A.E.O.L.U.S

FLIGHT CONTROLLER

Technical Design Report — Bottom-Up Approach

Robotics Society
Indian Institute of Technology Jodhpur

1. Introduction

The A.E.O.L.U.S project aims to design and develop a fully customisable quadcopter firmware with a robust flight control architecture. The bottom-up design approach starts from the physical hardware and mathematical plant model, progressively building up sensor estimation, control allocation, and high-level attitude/altitude controllers.

This report documents the bottom-up methodology exclusively, covering the system architecture, mathematical models, control design, sensor fusion, and simulation results obtained in MATLAB Simulink.

2. System Architecture

The bottom-up model is structured as three interconnected subsystems that mirror the real hardware stack:

Block	Role	Key Outputs
Plant	Simulates quadcopter rigid-body physics	Body-frame accelerations, angular rates, magnetometer readings
Estimation & Sensing	Discretises plant outputs; sensor fusion via complementary filter	Estimated roll, pitch, yaw
Controller	Joint attitude + altitude PID; motor mix allocation	Motor RPS commands

The top-level Simulink model below shows these three subsystems wired together, with the controller receiving estimated states and outputting motor speed commands back into the plant:

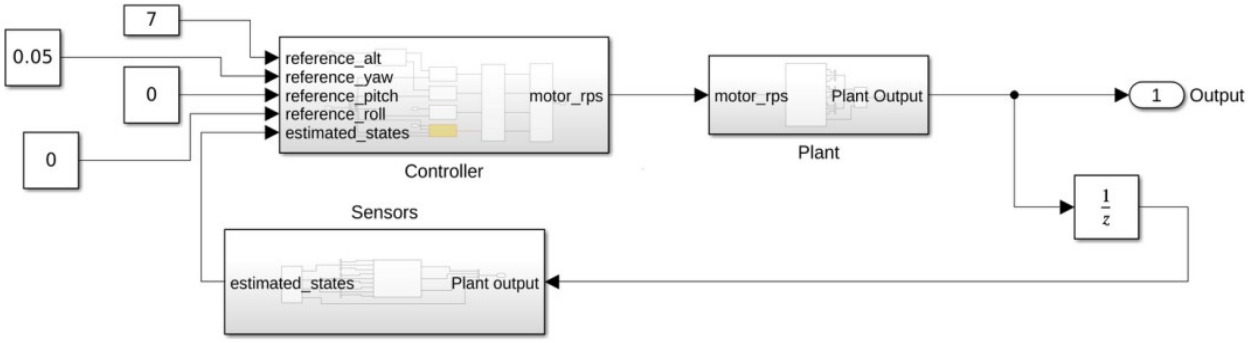


Figure 1 — Top-level Simulink model: Controller → Plant → Estimation feedback loop

3. Plant Model

The plant block simulates the physical dynamics of the quadcopter and outputs body-frame sensor readings that are fed into the estimation block. All dynamics are formulated in continuous time and subsequently discretised.

3.1 Thrust Generation

Each motor i produces thrust proportional to the square of its angular velocity:

$$T_i = b \cdot \omega_i^2 \quad (i = 1, 2, 3, 4)$$

where b is the thrust coefficient and ω_i is the angular speed of motor i.

3.2 Translational Dynamics

$$m \cdot \ddot{v} = [0, 0, mg]^T - R_BW \cdot [0, 0, T]^T$$

where R_BW is the rotation matrix from body to world frame and T is total thrust.

3.3 Rotational Dynamics

- Roll torque: $\tau_x = d(T_1 - T_2) = db(\omega_1^2 - \omega_2^2)$
- Pitch torque: $\tau_y = d(T_3 - T_4) = db(\omega_3^2 - \omega_4^2)$
- Yaw torque: $\tau_u = k(\omega_1^2 + \omega_3^2 - \omega_2^2 - \omega_4^2)$

$$J \cdot \dot{\omega} = -\omega \times J\omega + \Gamma$$

3.4 Plant Outputs

- Body-frame linear acceleration: $a_\beta = [\dot{u}, \dot{v}, \dot{w}]^T$
- Body-frame angular velocity: $\omega_\beta = [p, q, r]^T$
- Body-frame magnetometer: $m_\beta = R_BW\text{^}T \cdot m_w$

The Simulink plant subsystem below implements the full translational and rotational dynamics, outputting body-frame signals to the estimation block:

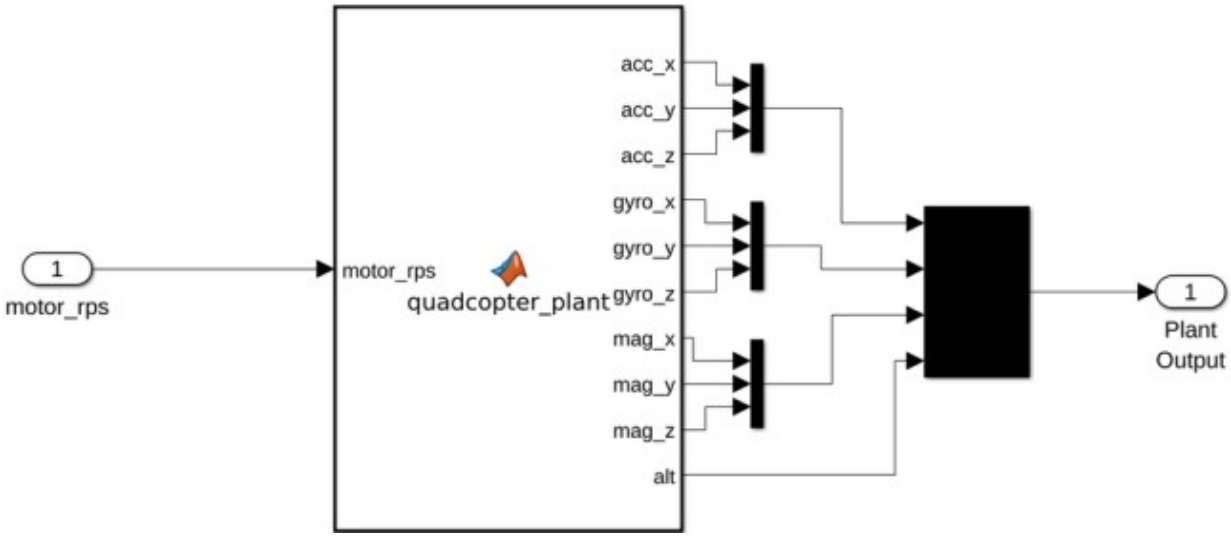


Figure 2 — Plant subsystem: rigid-body dynamics with translational and rotational equations

4. Control Allocation (Motor Mix)

The motor mix block converts desired torques (τ_x, τ_y, τ_u) and total thrust T into individual motor angular speeds using the control allocation matrix A:

	ω_1^2	ω_2^2	ω_3^2	ω_4^2
T	b	b	b	b
τ_x	0	−bd	0	bd
τ_y	−db	0	db	0
τ_u	k	−k	k	−k

The inverse A^{-1} exists provided b, k, d > 0. The Simulink implementation of the motor mix and its physical interpretation are shown below:

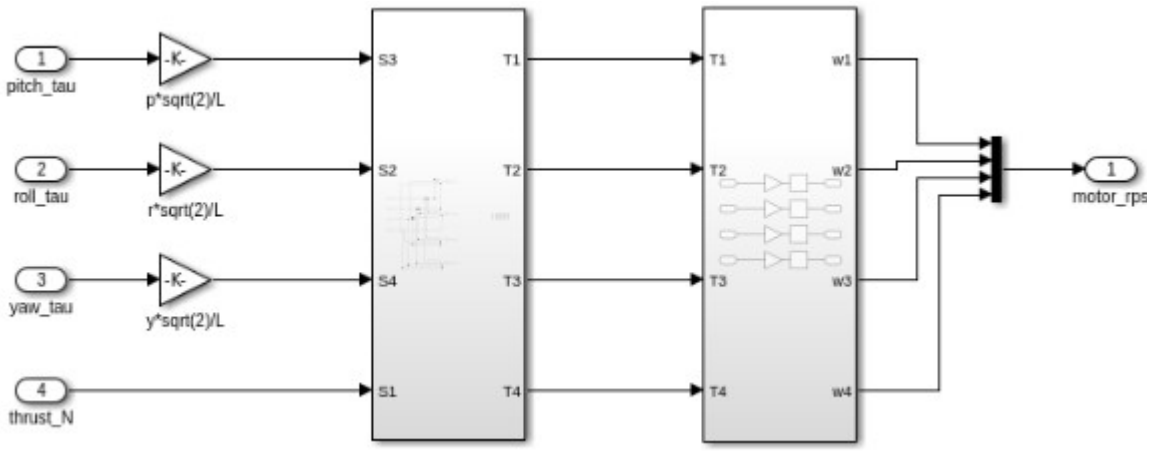


Figure 3 — Motor mix block: allocation matrix mapping torques and thrust to per-motor speeds

5. Estimation and Sensing

The estimation block receives continuous plant outputs, discretises them via rate transition, and fuses gyroscope, accelerometer, and magnetometer data through a complementary filter to produce reliable attitude estimates.

5.1 Gyroscope Integration

- roll_gyro(i) = prev_roll + $g_x \cdot dt$
- pitch_gyro(i) = prev_pitch + $g_y \cdot dt$
- yaw_gyro(i) = prev_yaw + $g_z \cdot dt$

The gyroscope integrates angular rates precisely over short intervals but accumulates drift over time.

5.2 Accelerometer Estimation

- roll_acc = $\arctan(a_y / a_z)$
- pitch_acc = $\arctan(-a_x / \sqrt{(a_y^2 + a_z^2)})$

Accelerometers are noise-sensitive and unreliable during dynamic manoeuvres.

5.3 Tilt-Compensated Magnetometer

- $m'_x = m_x \cdot \cos(\text{pitch_acc}) + m_z \cdot \sin(\text{pitch_acc})$
- $m'_y = m_x \cdot \sin(\text{roll_acc}) \cdot \sin(\text{pitch_acc}) + m_y \cdot \cos(\text{roll_acc}) - m_z \cdot \sin(\text{roll_acc}) \cdot \cos(\text{pitch_acc})$
- yaw_mag = $\arctan(-m'_y / m'_x)$

5.4 Complementary Filter

The complementary filter fuses the fast-but-drifting gyroscope with the stable-but-slow sensor estimates using a high-pass coefficient $\alpha = 0.9995$:

$$P_i = \alpha \cdot P_gyro + (1 - \alpha) \cdot P_acc$$

The same formula applies to roll, pitch (using accelerometer), and yaw (using magnetometer).

The Simulink estimation subsystem overview:

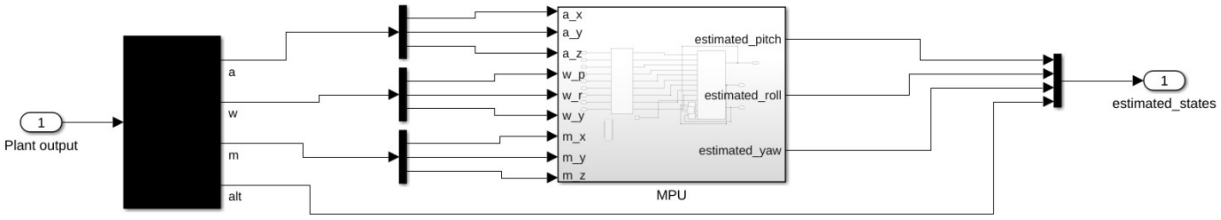


Figure 4 — Estimation subsystem: discretisation, sensor inputs, and MPU fusion block

6. Controller Design

The controller implements a joint attitude and altitude control framework. Pitch, roll, and yaw are treated as independent PID loops; altitude compensation accounts for tilt via the sec(θ) term.

6.1 Attitude PID Control

$$\tau_\theta(t) = K_p \cdot e_\theta(t) + K_i \cdot \int e_\theta(\tau) d\tau + K_d \cdot (de_\theta/dt)$$

6.2 Altitude PID with Attitude Compensation

- Altitude error: $e_u = z_ref - z_est$
- PID thrust: $T_PID = PID_z(e_u)$
- Total thrust: $T_total = (mg + T_PID) \cdot \sec(\theta_p) \cdot \sec(\theta_r)$
- Motor thrust: $T_thrust = T_total$ (if $T_total \geq 0$, else 0)

The $\sec(\theta_p) \cdot \sec(\theta_r)$ term compensates for tilt, maintaining vertical thrust regardless of attitude.

The Simulink controller subsystem below shows the parallel PID channels for pitch, roll, yaw, and altitude feeding into the motor mix:

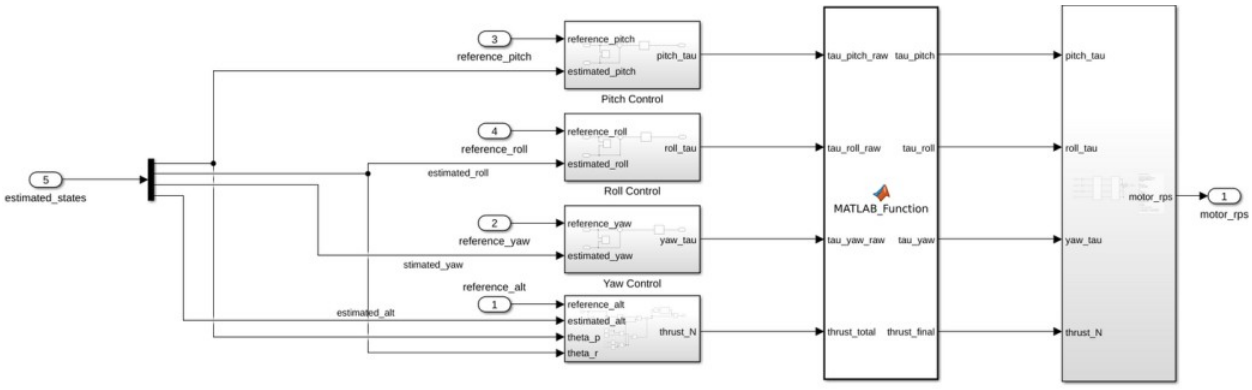


Figure 5— Controller subsystem: independent attitude PIDs and altitude PID with tilt compensation

6.3 Control Architecture Summary

Loop	Input	Output	Method
Roll PID	Ref. roll, estimated roll	Roll torque τ_x	PID
Pitch PID	Ref. pitch, estimated pitch	Pitch torque τ_y	PID
Yaw PID	Ref. yaw, estimated yaw	Yaw torque τ_u	PID
Altitude PID	Ref. altitude, estimated alt.	Thrust T (tilt-compensated)	PID + sec(θ)
Motor Mix	τ_x, τ_y, τ_u, T	ω_1, ω_2, ω_3, ω_4	A^{-1} allocation

7. Simulink Implementation

The complete system is modelled in MATLAB Simulink using a hierarchical block structure. Each subsystem (Plant, Controller, Estimation/Sensing) is encapsulated separately and connected by bus signals.

- The Plant subsystem integrates translational and rotational dynamics using a fixed-step ode4 (Runge-Kutta) solver.
- Rate Transition blocks bridge continuous plant outputs and discrete sensor/estimation logic running at the IMU sample rate.
- The Controller subsystem houses independent PID blocks for pitch, roll, and yaw, plus the altitude PID with tilt-compensation MATLAB Function.
- The Motor Mix subsystem implements A^{-1} allocation to convert control torques to per-motor speed commands.

8. Simulation Results

The bottom-up Simulink model was simulated with simultaneous altitude and attitude reference inputs including dynamic reference altitude via gamepad. The scope outputs below capture the system response:



Figure 6 — Simulation results: altitude tracking (left), yaw attitude tracking (centre), dynamic reference altitude control via gamepad (right)

Test Scenario	Observation
Step altitude reference	Drone climbs to target; attitude-invariant thrust maintains level flight during ascent.
Simultaneous yaw + altitude	Altitude and yaw channels show decoupled tracking; small cross-coupling transients decay quickly.
Dynamic reference altitude (gamepad)	Controller tracks smoothly varying reference; tilt compensation effective across attitude envelope.
Motor power distribution	All four motors operate within valid RPM bounds; allocation matrix correctly distributes torques.

The complementary filter successfully corrects gyroscope drift while rejecting high-frequency accelerometer noise, producing stable attitude estimates throughout the simulation.

9. Conclusion

The bottom-up design approach for the A.E.O.L.U.S flight controller demonstrates a rigorous, physically grounded methodology for quadcopter firmware development. Beginning from first-principles rigid-body dynamics and progressively adding sensor fusion, control allocation, and coupled PID loops results in a modular, verifiable architecture.

The Simulink model achieves simultaneous altitude and attitude control with attitude-invariant thrust compensation, validated against step and dynamic reference inputs. The complementary filter provides reliable IMU fusion without the computational overhead of a Kalman filter, making it suitable for embedded MCU implementation.

Future work includes hardware-in-the-loop testing on a real flight controller, PID gain tuning for physical motor/ESC dynamics, and GPS-based position hold integration.